ICLR 2026

Efficient Reasoning with Balanced Thinking

REBALANCE: A Training-Free Framework for Dynamic Reasoning Control in Large Reasoning Models

Training-Free | Plug-and-Play | Generalizable

Presentation Overview

01 **Introduction & Problem Statement**
Understanding overthinking and underthinking challenges in LRMs

02 **Background & Motivation**
Foundational concepts: stepwise confidence and variance metrics

03 **Key Observations**
Confidence as a reliable indicator of reasoning states

04 **Method Overview**
REBALANCE framework architecture and pipeline

05 **Explicit Modeling**
Formal definitions and classification of reasoning states

06 **Confidence-Based Steering**
Prototype extraction and steering vector construction

07 **Dynamic Control Function**
Adaptive modulation of steering strength and direction

08 **Experimental Setup**
Benchmarks, models, and baseline comparisons

09 **Main Results**
Performance analysis on math reasoning benchmarks

10 **Cross-Domain Generalization**
Performance beyond mathematical reasoning tasks

11 **Ablation Studies**
Component-wise analysis and sensitivity testing

12 **Conclusion & Future Work**
Key contributions and research directions

CHAPTER 01

Introduction & Problem Statement

Understanding the challenges of efficient reasoning in Large Reasoning Models

PROBLEM 01

The Overthinking Problem in LRMs

What is Overthinking?

Large Reasoning Models (LRMs) expend redundant computational steps on simple problems, continuing to reason even after reaching the correct answer.

> **Example: A model correctly solves a math problem in Step 5, but continues generating 15+ additional verification steps**

↑ 300%

Token Overhead

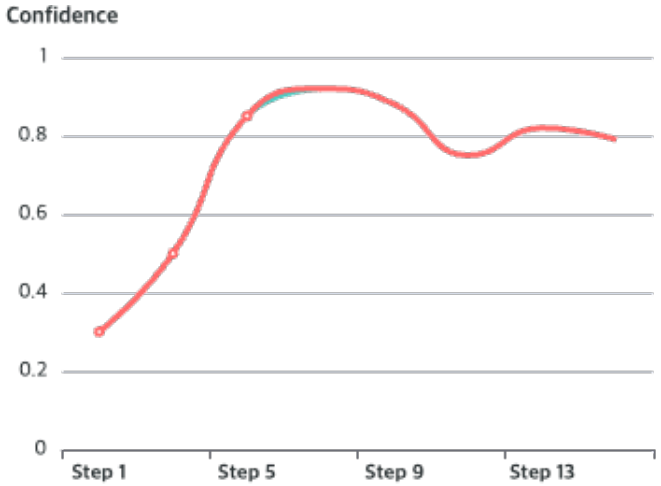
vs. optimal reasoning length

~2%

Accuracy Gain

marginal improvement

Reasoning Length Distribution



Consequences of Overthinking

1. **Computational Waste:** Excessive token generation
2. **Hallucinations:** More steps → more errors
3. **Latency Issues:** Slower response times
4. **Deployment Barriers:** Resource constraints

Key Insight

Overthinking severely limits **practical deployment** of LRMs in resource-constrained environments, creating an urgent need for efficient reasoning control.

PROBLEM 02

The Underthinking Challenge

What is Underthinking?

LRMs fail to sufficiently explore valid reasoning paths despite possessing the inherent capability to solve the problem.

> Example: A model commits to an incorrect answer in Step 3, when the correct path would require 2-3 more exploration steps

Current Mitigation Approaches

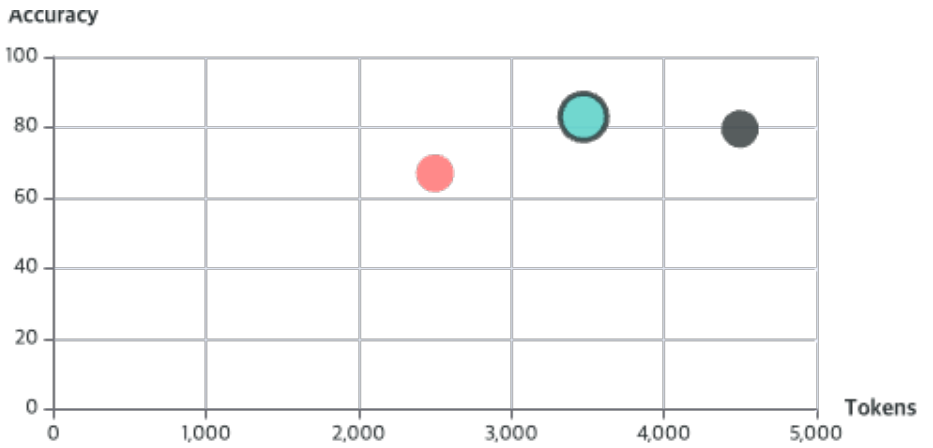
A Keyword Suppression

Suppress reflective keywords ("wait", "let me check"). But this **indiscriminately affects** both redundant AND valuable reasoning.

B Length-Based Adjustment

Adjust reasoning length via SFT/RL. But this often **penalizes lengthy reasoning** or dilutes rewards for control tokens.

The Trade-off Problem



The Critical Issue

Existing methods that mitigate **overthinking** often introduce **underthinking** as an unintended side effect.

→ We need a method that can handle BOTH problems simultaneously

CENTRAL QUESTION

How can we mitigate overthinking without inducing underthinking ?

Achieving efficient reasoning with balanced thinking

Current Limitation

Rigid binary selection (keep/discard entire paths) sacrifices potentially valuable intermediate reasoning steps

Our Insight

Need a continuous and reliable indicator of reasoning states for dynamic fine-grained control

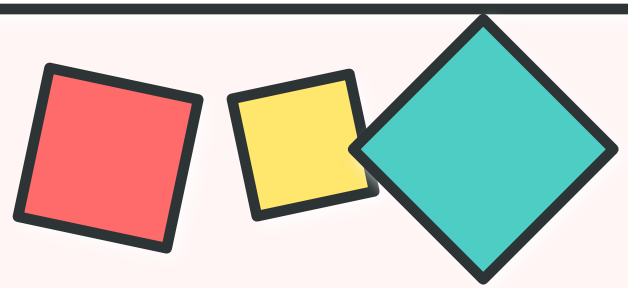
Our Solution

Confidence-based dynamic control that adapts to real-time reasoning states

CHAPTER 02

Background & Motivation

Foundational concepts and preliminary observations



PRELIMINARY 01

Stepwise Confidence: Measuring Reasoning Consistency

Definition

Stepwise confidence **c_s** measures the degree to which the model consistently adheres to the same reasoning path at step **s**.

Mathematical Formulation:

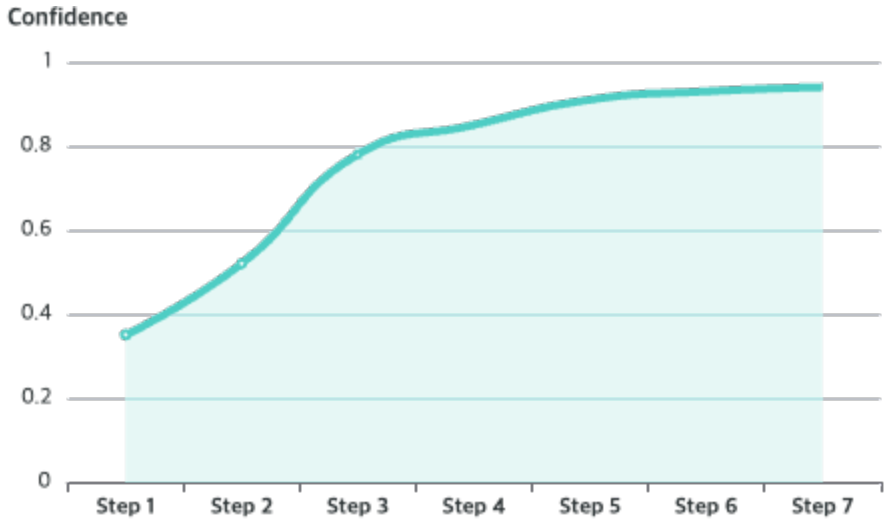
$$c_s = \exp\left((1/|T_s|) \times \Sigma_{t \in T_s} \ln(p_{max}^t) \right)$$

where **p_max^t** = max probability at token position t, and **|T_s|** is the number of tokens in step s

Key Insight

Higher confidence indicates the model is more certain about its current reasoning direction, while lower confidence suggests hesitation or uncertainty.

Confidence Evolution During Reasoning



↑ High

Consistent Path

Model is certain

↓ Low

Uncertain Path

Model is hesitant

PRELIMINARY 02

Confidence Variance: Capturing Path Switching

Definition

Confidence variance **Var(c_s; W_s)** captures short-term fluctuations in confidence within a sliding window, quantifying the frequency of switching between different reasoning paths.

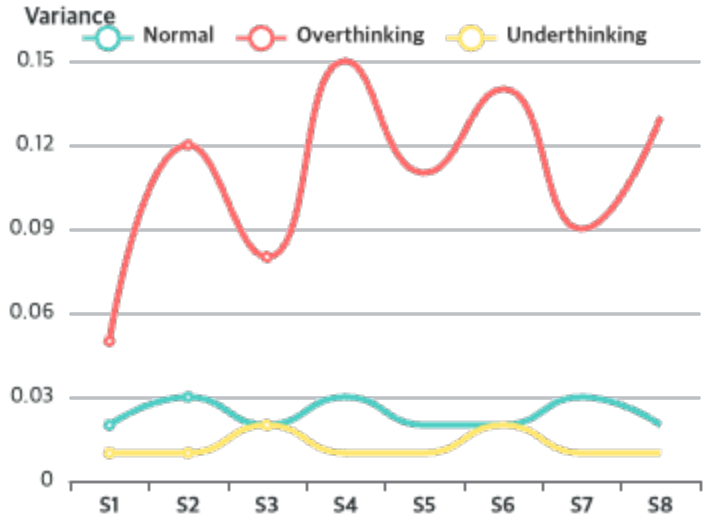
Mathematical Formulation:

$$W_s = \{\max(1, s-W+1), \ldots, s\}$$

$$\mathrm{Var}(c_s; W_s) = (1/|W_s|) \times \Sigma_{j \in W_s} (c_j - \bar{c}_s)^2$$

where **W_s** is the sliding window and **c̄_s** is the average confidence within the window

Variance Patterns



↑ High Variance

Frequent switching between paths → **Overthinking**

↓ Low Variance

Stable confidence → Normal reasoning or **Underthinking**

Window Size |W|

Typical window size: **|W| = 5** . Larger windows smooth short-term fluctuations but may miss local patterns.

KEY FINDING

Confidence as a Reasoning Indicator

Empirical Observation

Confidence values correlate strongly with LRMs' reasoning behaviors, making it a reliable continuous indicator.

High Confidence Variance

Reflects **indecisive switching** between different reasoning paths → Overthinking

Consistent Overconfidence

Leads to **premature commitment** to incorrect paths without exploration → Underthinking

Why This Matters

Confidence enables fine-grained behavioral control, allowing dynamic adjustments during the reasoning process.

Reasoning State Classification

Low	High	Med
Confidence + High Variance	Confidence + Low Variance	Confidence Normal Range

CHAPTER 03

Method
Overview

The REBALANCE framework architecture

FRAMEWORK

REBALANCE Framework Architecture

1 Explicit Modeling

Identify reasoning steps indicating **overthinking** and **underthinking** using confidence metrics.

Key Operations:

- Compute stepwise confidence
- Calculate confidence variance
- Classify reasoning states
- Define thresholds using quantiles

2 Steering Vector

Extract prototypical representations from **deep-layer hidden states** to construct steering direction.

Key Operations:

- One-pass prototype extraction
- Aggregate overthinking prototypes
- Aggregate underthinking prototypes
- Compute normalized steering vector

3 Dynamic Control

Modulate steering **strength and direction** based on real-time confidence during inference.

Key Operations:

- Monitor confidence & variance
- Compute steering direction δ_s
- Compute steering strength λ_s
- Apply to hidden states online

Uses small seen dataset

Captures reasoning dynamics

Adaptive real-time control

Training-Free Plug-and-Play Generalizable No Extra Forward Passes

PIPELINE

Offline vs Online Processing

OFFLINE STAGE

One-time setup

1. **Data Collection**
   Perform **one-pass inference** on small seen dataset (500 MATH problems)
2. **State Classification**
   Segment reasoning steps and classify into **O (overthinking)** and **U (underthinking)** sets
3. **Prototype Extraction**
   Extract hidden states from **deep layers** and aggregate into prototypes O and U
4. **Control Function Fitting**
   Fit **dynamic control function** g(cs, vs) based on model behavior

Executed once per model

ONLINE STAGE

Real-time inference

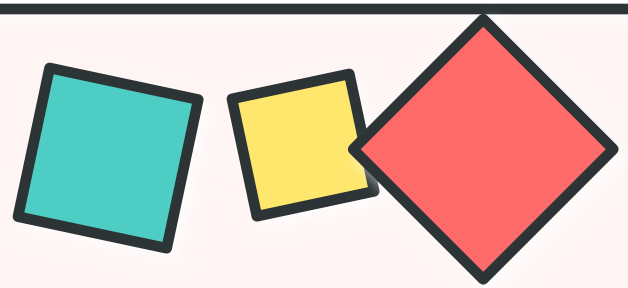
1. **Confidence Monitoring**
   At each step s, compute **real-time confidence cs** and **variance vs**
2. **Steering Weight Computation**
   Apply control function: **$\alpha_s = g(c_s, v_s)$** to get direction and strength
3. **Hidden State Modulation**
   Inject steering: **$\tilde{h} = h + \alpha_s \cdot v$** at first token of selected layer
4. **Continue Generation**
   Generate next tokens using modulated hidden states, **no extra forward passes**

Executed for every inference

CHAPTER 04

Explicit Modeling of Reasoning States

Formal definitions and classification criteria



DEFINITIONS

Formal Definitions: Overthinking & Underthinking

OVERTHINKING

Stability Index

$$s^* = \min\{s : d_{s'} = d_s \text{ for all } s' \geq s \text{ and } d_s \text{ is correct}\}$$

The **stability index s^*** is the earliest step where the model converges to the correct answer and maintains it.

Overthinking Condition

A trajectory exhibits overthinking if it continues generating steps after reaching stability ($s > s^*$).

Captures redundant computation after convergence

UNDERTHINKING

Premature Termination

Underthinking Condition:

A trajectory exhibits underthinking if it stops at step s with incorrect prediction d_s while there exists $s' > s$ with correct $d_{s'}$.

The model **prematurely commits** to an incorrect answer before exploring sufficient reasoning paths.

Characteristics

- **High confidence** in incorrect answer
- **Low variance** (no exploration)
- Stops before reaching s^*
- Model has capability but doesn't use it

Captures premature termination before sufficient reasoning

CLASSIFICATION

Classification Using Confidence Thresholds

Threshold Computation

Using empirical quantiles from a small seen dataset, we compute thresholds for confidence and variance:

$\tau_c^L = Q_c(q_L)$
$\tau_c^H = Q_c(q_H)$

$\tau_v^L = Q_v(q_L)$
$\tau_v^H = Q_v(q_H)$

where $0 < q_L < q_H < 1$ specify lower and upper quantiles (typically $q_L=0.2$, $q_H=0.8$)

State Classification

O Overthinking Set

$O \leftarrow \{s : c_s \leqslant \tau_c^L \wedge v_s \geqslant \tau_v^H\}$

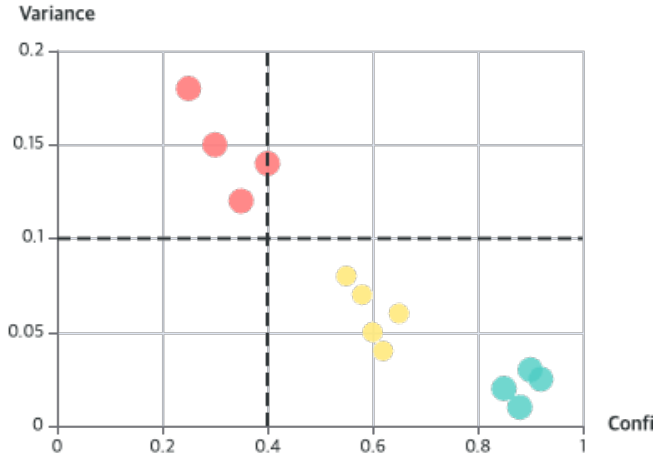
Low confidence + High variance → unstable, oscillating trajectories

U Underthinking Set

$U \leftarrow \{s : c_s \geqslant \tau_c^H \wedge v_s \leqslant \tau_v^L\}$

High confidence + Low variance → premature convergence

Confidence-Variance Space



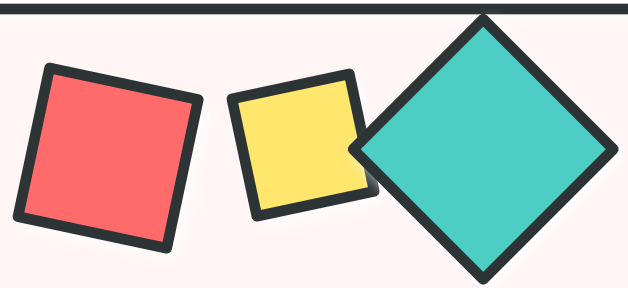
Normal Reasoning

Instances **not in O ∪ U** are treated as normal and excluded from further analysis.

CHAPTER 05

Confidence-Based Steering Vector

Extracting and constructing steering vectors



EXTRACTION

One-Pass Prototype Extraction

Offline Process

1 Single Inference Pass

Perform **one forward pass** over small seen dataset D_seen (500 MATH problems)

2 Step Segmentation

Segment reasoning trajectory by **delimiter "\n\n"** to identify individual steps

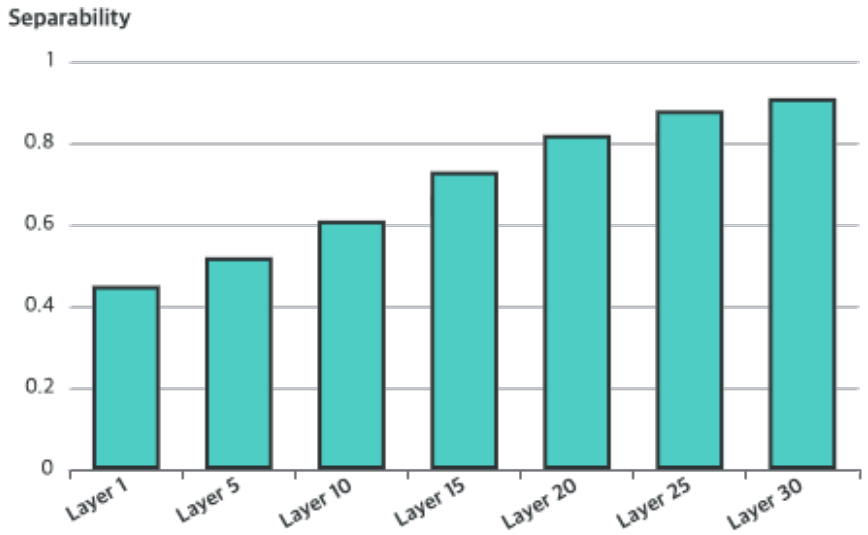
3 Layer Selection

Automatically select **optimal deep layer** based on intrinsic separability of reasoning modes

4 Hidden State Collection

Collect hidden states **h_t(1)_s** at first token of each step from selected layer

Layer Selection Analysis



Why Deep Layers?

Deeper layers exhibit **stronger discriminability** between reasoning modes and improved generalization across datasets.

First Token Selection

h_t(1)_s serves as compact encoding of **step-level intent** and conditions generation of all subsequent tokens within the step.

CONSTRUCTION

Prototype Aggregation & Steering Vector

Prototype Aggregation

Aggregate hidden states from classified steps to create prototypical representations:

O Overthinking Prototype

$$\mu_O = (1/|O|) \times \Sigma_{s \in O} h_t(1)_s$$

U Underthinking Prototype

$$\mu_U = (1/|U|) \times \Sigma_{s \in U} h_t(1)_s$$

Prototype Properties

- Capture **model's inherent reasoning dynamics**
- Exhibit **strong generalization** across datasets
- Enable **fine-grained behavior control**

Steering Vector Construction

The steering vector represents the **direction from underthinking to overthinking** :

$$v = (\mu_O - \mu_U) / \|\mu_O - \mu_U\|_2$$

↑ Positive Direction (+v)

Stimulates **exploration** → addresses underthinking

↓ Negative Direction (-v)

Encourages **commitment** → mitigates overthinking

The steering vector captures the model's inherent reasoning dynamics

MODULATION

Hidden State Modulation Mechanism

Modulation Formula

At each reasoning step, adjust the hidden state at the first token:

$$\hat{h}_{t(1)\_s} = h_{t(1)\_s} + \alpha_s \cdot v$$

Where:

- **$h_{t(1)\_s}$** : original hidden state at step s
- **v** : steering vector (normalized)
- **α_s** : signed steering weight at step s

Steering Weight Components

$$\alpha_s = \lambda_s \cdot \delta_s$$

$\lambda_s \geq 0$
Steering strength → magnitude of adjustment

$\delta_s \in \{+1, -1\}$
Steering direction → exploration vs commitment

Directional Effects

+1 $\delta_s = +1$

Stimulates Exploration

- Addresses underthinking
- Encourages alternative paths
- Increases reasoning diversity

-1 $\delta_s = -1$

Encourages Commitment

- Mitigates overthinking
- Reduces redundant steps
- Promotes faster convergence

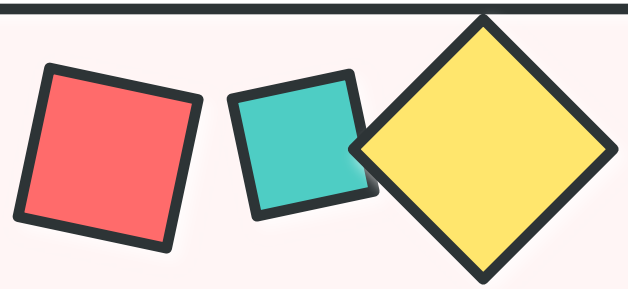
Establishes boundaries for balanced reasoning

CHAPTER 06

Dynamic Control

Function

Adaptive modulation of steering strength and direction



CONTROL

Dynamic Control Function Design

Function Definition

The dynamic control function **g(c_s, v_s)** outputs steering weight based on current confidence and variance:

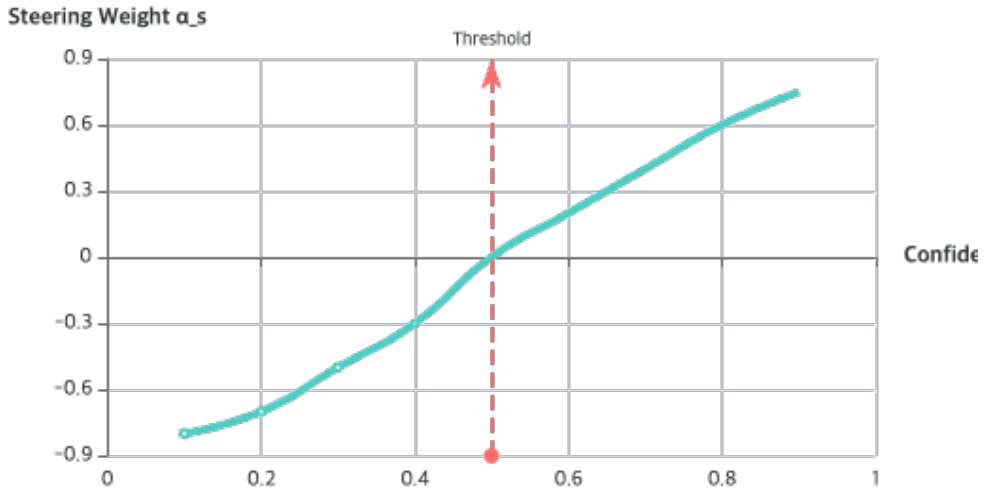
$$\alpha_s = g(c_s, v_s) = \delta_s \cdot \lambda_s$$

This adaptive mechanism ensures trajectories stay between overthinking and underthinking boundaries.

Online Application

1. At each step s, obtain **c_s** and **v_s** from model
2. Compute **α_s = g(c_s, v_s)** using fitted function
3. Inject **α_s · v** at first token of selected layer
4. **No extra forward passes** beyond standard decoding

Control Function Visualization



Prune

Overthinking

α_s < 0

Explore

Underthinking

α_s > 0

COMPONENTS

Steering Direction & Strength

STEERING DIRECTION

$$\delta_s = \text{sign}(c_s - \tau_{c^\wedge H})$$

The direction is determined by comparing current confidence to the high-confidence threshold:

− $c_s < \tau_{c^\wedge H} \rightarrow \delta_s = -1$

Confidence below threshold → **mitigate overthinking** by encouraging commitment

+ $c_s > \tau_{c^\wedge H} \rightarrow \delta_s = +1$

Confidence above threshold → **alleviate underthinking** by stimulating exploration

Guarantees steering directs away from nearer boundary

STEERING STRENGTH

$$\lambda_s = |c_s - \tau_{c^\wedge H}| \cdot B(c_s, v_s) \cdot \tanh(|c_s - \tau_{c^\wedge H}|)$$

The strength combines two components for smooth, adaptive control:

Soft Saturation

$\tanh(|c_s - \tau_{c^\wedge H}|)$ ensures smooth, saturating growth

- Avoids abrupt changes
- Maintains monotonicity
- Ensures numerical stability

Variance-Aware Amplitude

$B(c_s, v_s)$ adapts based on model behavior

- Indicates current reasoning status
- Shifts between mode boundaries
- Model-specific adaptation

AMPLITUDE

Variance-Aware Amplitude Function

Piecewise Formulation

The amplitude function shifts between mode boundaries based on current reasoning state:

$$B(c_s, v_s) = B_m + (B_o - B_m) \cdot \psi(c_s, v_s)$$
if $c_s \leq \tau_c^L$ and $v_s \geq \tau_v^H$ (overthinking)

$$B(c_s, v_s) = B_m + (B_u - B_m) \cdot \psi(c_s, v_s)$$
if $c_s \geq \tau_c^H$ and $v_s \leq \tau_v^L$ (underthinking)

$$B(c_s, v_s) = B_m$$ otherwise
(moderate reasoning)

Gating Function

$\psi(c_s, v_s)$ is a conditioned gating function ensuring smooth transitions:

- Output range: $[0, 1]$
- Ensures **smooth transitions** between modes
- Default: **sigmoid gate** (robust to alternatives)

Mode Boundaries

B_m (Moderate)
Baseline amplitude for normal reasoning

B_o (Overthinking)
Amplitude boundary for overthinking mode

B_u (Underthinking)
Amplitude boundary for underthinking mode

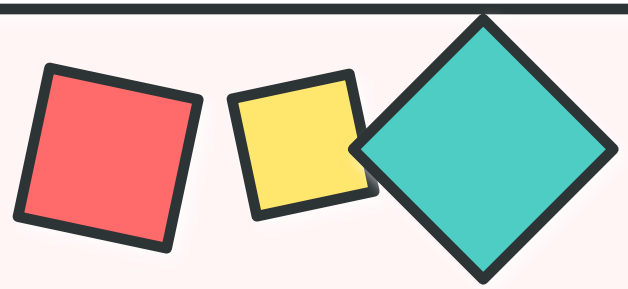
B_m and B_o are adaptively derived from model behavior without manual tuning

Amplitude indicates current reasoning status

CHAPTER 07

Experimental Setup

Evaluation methodology and benchmarks



SETUP

Evaluation Benchmarks & Models

BENCHMARKS

Mathematical Reasoning

- MATH-500
- AIME24
- AIME25
- AMC23
- GSM8K
- OLYMPIAD

Scientific Reasoning

- GPQA-DIAMOND

Commonsense & Code

- STRATEGYQA
- LIVECODEBENCH

MODELS

DeepSeek-R1-Distill-Qwen

- 1.5B
- 7B

Qwen3

- 14B

QwQ

- 32B

Scale: 0.5B to 32B parameters

500 MATH problems for steering extraction

Held fixed across all unseen benchmarks

COMPARISON

Baseline Methods Comparison

Prompt-Based Suppression	Length-Based Adjustment	External Verifier	Intrinsic Signal
NoThinking Suppress reflective keywords	**DEER** Dynamic Early Exit	**FlashThink** Early exit method	**Manifold Steering** Latent space control
NoWait Remove "wait" tokens	**Dynasor-CoT** Adaptive CoT length	**TrimR** Verifier-based compression	**REBALANCE** Confidence-based dynamic
CoD Chain of Draft	**SEAL** Steerable calibration		
⚠ May cause underthinking	⚠ May penalize long reasoning	⚠ External criteria may mismatch	✓ Intrinsic & reliable

🏆 REBALANCE uniquely balances overthinking and underthinking through confidence-based dynamic control

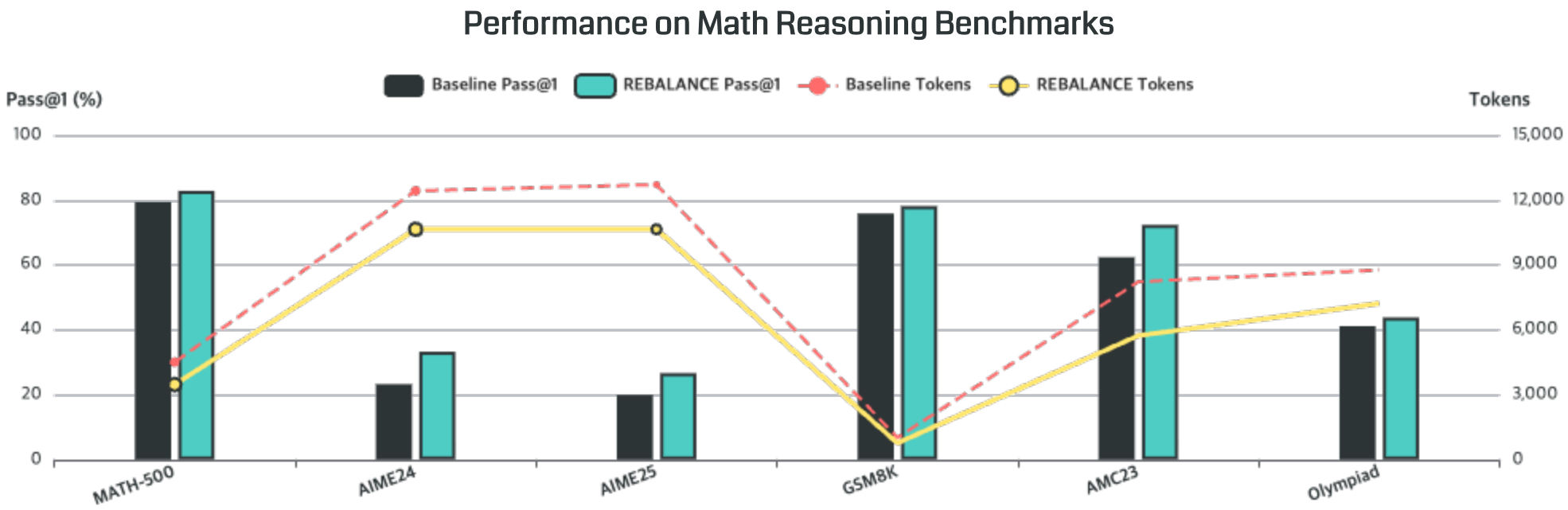
CHAPTER 08

Main Results:
Math Reasoning

Performance analysis on mathematical reasoning benchmarks

RESULTS 01

Overall Performance: DeepSeek-R1-Distill-Qwen



+10.0
Max Accuracy Gain
points improvement

-35.4%
Max Token Reduction
efficiency gain

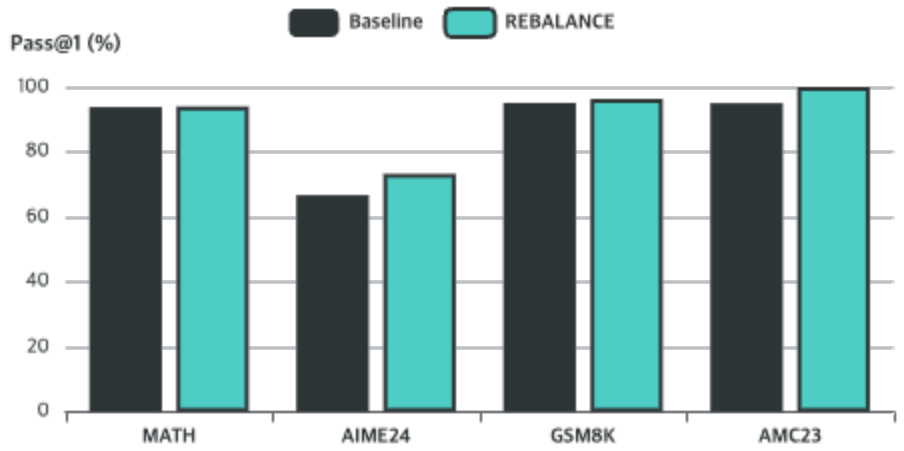
6
Benchmarks
comprehensive eval

SOTA
vs. All Baselines
consistent wins

RESULTS 02

Overall Performance: Qwen3-14B & QwQ-32B

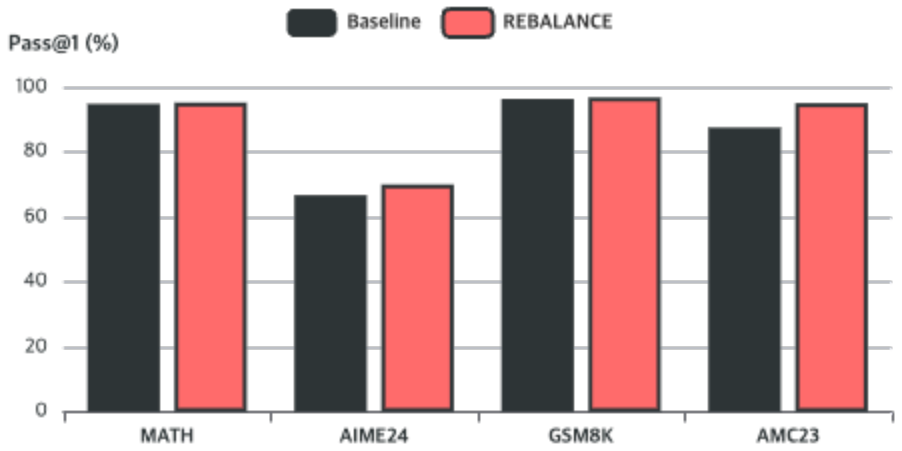
Qwen3-14B Results



+6.6%
Max Accuracy

-13.1%
Token Reduction

QwQ-32B Results



+10.0%
Max Accuracy

-27.8%
Token Reduction

REBALANCE maintains effectiveness across different model families and scales (0.5B to 32B)

ANALYSIS

Key Behavioral Patterns Across Models

Pattern 01: Prompt Sensitivity

Prompt-based suppression methods (NoThinking, NoWait) affect distilled models much more severely than non-distilled ones.

Distilled Models (R1-Distill)

- **Large accuracy drops** observed
- More sensitive to prompt-level manipulations
- Reasoning structure heavily affected

Non-Distilled Models (Qwen3, QwQ)

- **Comparatively modest** impact
- More robust to prompt changes
- Better preserve reasoning capabilities

Implication: Distilled models require more careful handling

Pattern 02: Signal Source

Methods relying on **external auxiliary models** for early-exit decisions generally underperform compared to approaches based on **intrinsic model signals** .

External Verifiers (TrimR, FlashThink)

- May not reflect **internal reasoning state**
- Additional computational overhead
- Potential mismatch with target model

Intrinsic Signals (DEER, REBALANCE)

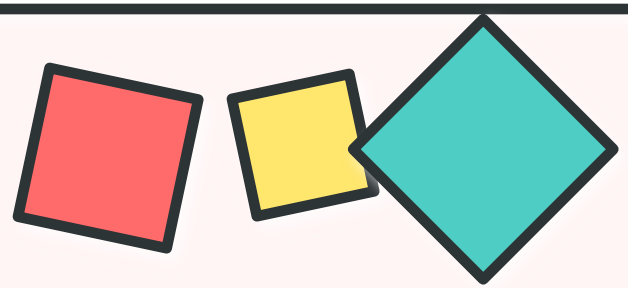
- More **reliable and lightweight**
- Faithfully reflect internal state
- No additional models needed

Intrinsic signals provide superior control mechanism

CHAPTER 09

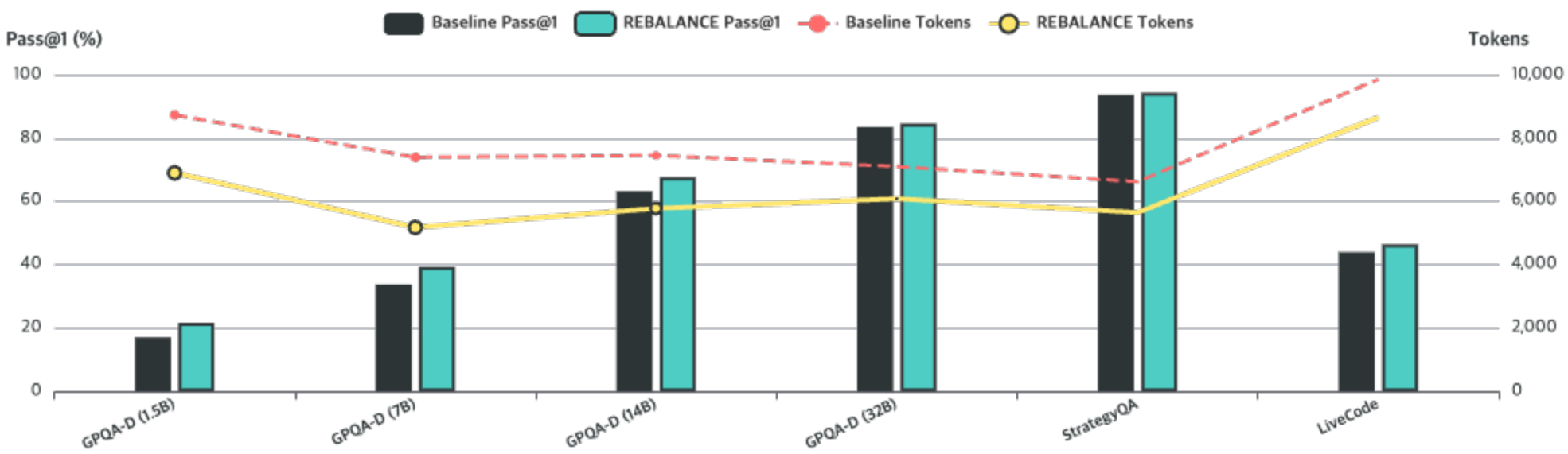
Cross-Domain Generalization

Performance beyond mathematical reasoning



GENERALIZATION

Generalization to Non-Math Tasks



GPQA-D
Science
+5.6% acc, -22.4% tokens

LiveCode
Programming
+2.5% acc, -12.2% tokens

StrategyQA
Commonsense
+1.1% acc, -14.3% tokens

Max
Token Reduction
-29.9% on GPQA-D

BEHAVIOR

Task-Specific Behavior Analysis

Challenging Tasks

GPQA, Code Generation

Performance

- **Significant improvement** in effective reasoning
- Substantial **reduction in redundancy**
- Better exploration of solution space

Behavior

- Model exhibits **more overthinking states**
- Method effectively prunes redundancy
- Maintains accuracy while reducing tokens

Strong improvement

Saturated Tasks

StrategyQA

Performance

- **Modest gains** in accuracy
- **Conservative behavior** observed
- Smaller token reduction

Behavior

- Model exhibits **fewer overthinking states**
- Already near optimal reasoning
- Less room for improvement

Conservative approach

Key Insight

Confidence Generalization

The **confidence signals** extracted from mathematical reasoning generalize well across different task domains.

Why It Works

Reasoning dynamics (confidence patterns) are **task-agnostic** and capture fundamental model behavior

Practical Impact

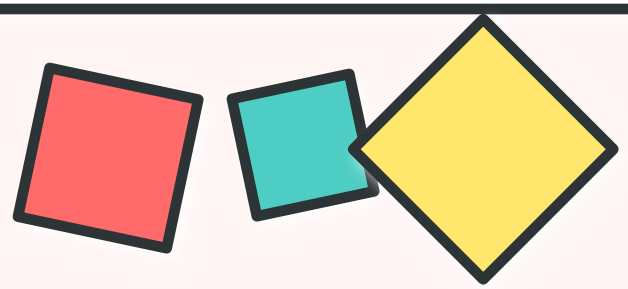
No need for **domain-specific tuning** → plug-and-play deployment

Strong cross-domain generalization

CHAPTER 10

Ablation Studies

Component-wise analysis and sensitivity testing



ABLATION 01

Impact of Static vs Dynamic Control

Experimental Setup

Compare fixed steering weights (α_s) versus dynamic control on MATH-500:

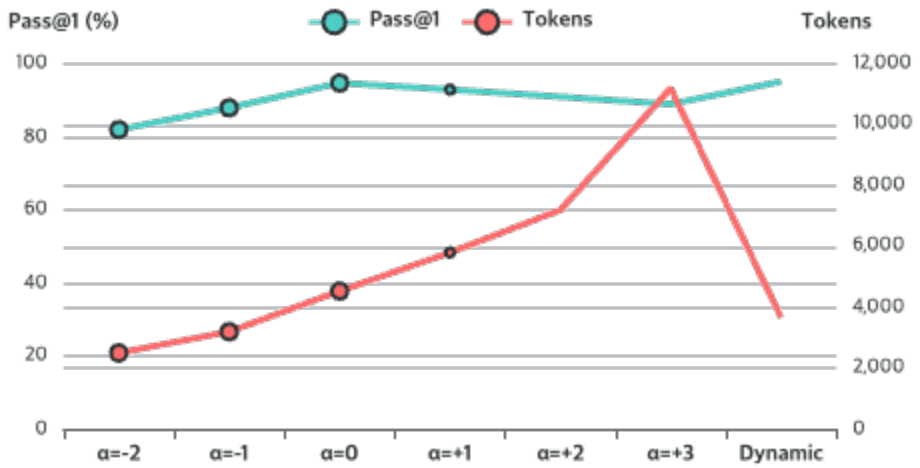
Positive α_s (U → O)
Stimulates exploration → **improves accuracy** but increases tokens

Negative α_s (O → U)
Encourages commitment → **reduces tokens** but decreases accuracy

Key Finding

Static control cannot adapt to instance difficulty, motivating the need for dynamic adaptation .

Static α_s Control (QwQ-32B)



α_s = +3
+147% tokens
at accuracy expense

Dynamic
Optimal balance
adapts per instance

ABLATION 02

Steering Layer Selection Analysis

Layerwise Performance

Test generalization by fitting steering vectors and control surfaces at various layers (selected by depth ratio):

Key Findings

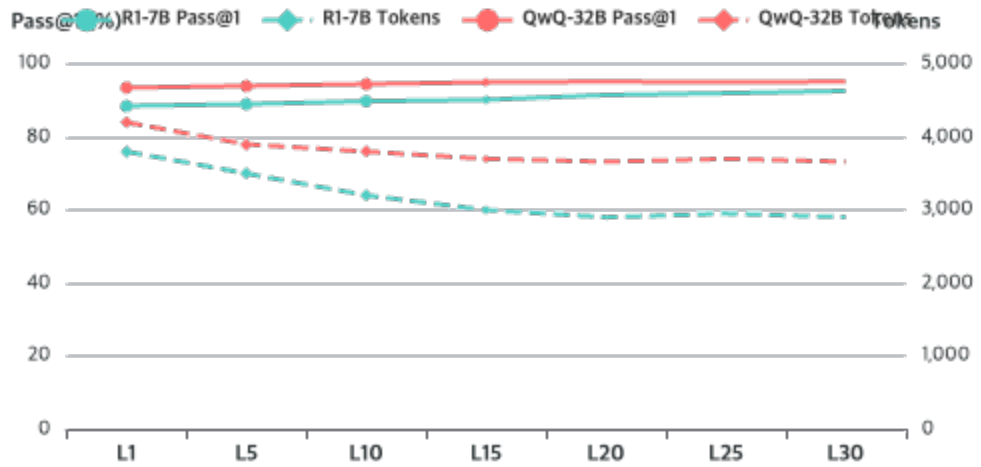
- Steering at **any tested depth** reduces tokens
- **No accuracy degradation** across layers
- Strongest trade-off in **mid-to-late layers**

Why Mid-to-Late Layers?

Representations from these layers exhibit the **highest confidence separability** and thus incur minimal noise

Consistent with probing analysis (Appendix A.5)

Layer Selection (R1-7B & QwQ-32B)



Early Layers
Lower separability

Mid-Late Layers
Optimal balance

ABLATION 03

Steering Vector Generalization

Cross-Dataset Transfer

Analyze vectors estimated from math datasets of varying scales and cross-domain science corpus (GPQA):

Key Findings

- Vectors **generalize across datasets**
- Improve efficiency while **preserving accuracy**
- No domain-specific tuning needed

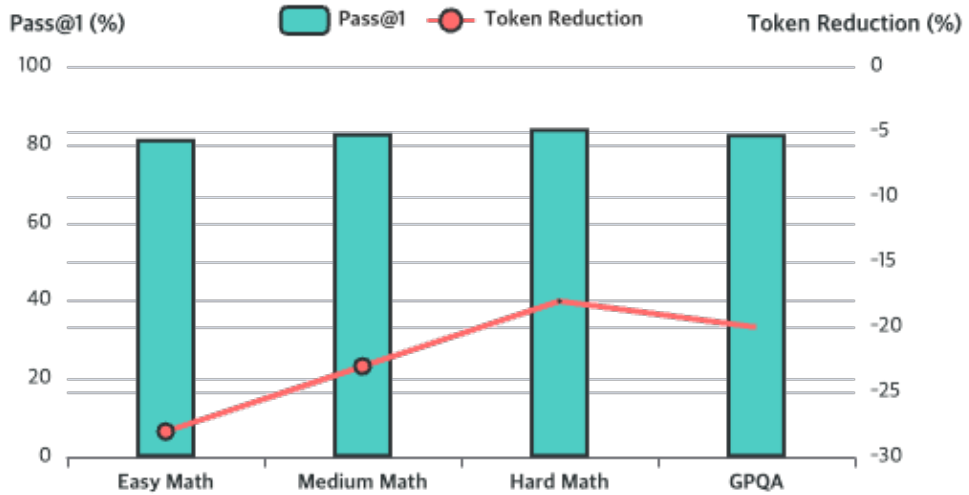
Clear Trend Emerges

Vectors from **harder datasets** prioritize accuracy gains over token savings

Harder data → conservative surface (prioritizes correctness)
Easier data → aggressive surface (favors token savings)

Aligns with method's mechanism

Vector Source Impact



Easy Math
Aggressive

Hard Math
Conservative

GPQA
Science

ABLATION 04

Window Size & Underthinking Boundary

Window Size |W_s|

Analyze how control-surface window size affects reasoning:

Larger Windows

- **Substantially increase** token usage
- Smooth short-term fluctuations
- **Reduce responsiveness** to local anomalies
- Improve accuracy on **hard benchmarks**

Smaller Windows

- More **sensitive to local patterns**
- Satisfies **Markovian continuity**
- Default: **|W_s| = 5** (optimal balance)

Underthinking Boundary B_u

Analyze impact of underthinking mode boundary:

Removing B_u (B_u = 0)

- Slightly **reduces tokens**
- **Significantly lowers accuracy**
- Especially on tasks requiring **extended reasoning**

Moderate Increase (B_u = 0.2)

- Encourages **deeper deliberation**
- **Boosts accuracy** at modest token cost
- Optimal balance achieved

Large Increase (B_u = 0.5)

- Triggers **overthinking**
- **Degrades performance**
- Disrupts normal reasoning paths

Extended deliberation improves hard task performance

B_m and B_e adaptively derived (no manual tuning)

ABLATION 05

Gating Function Robustness

Robustness Testing

Investigate sensitivity to specific form of gating function by replacing default sigmoid with alternatives:

1. Sigmoid Gate (Default)
Smooth S-shaped curve

2. Linear Gate
Straight-line transition

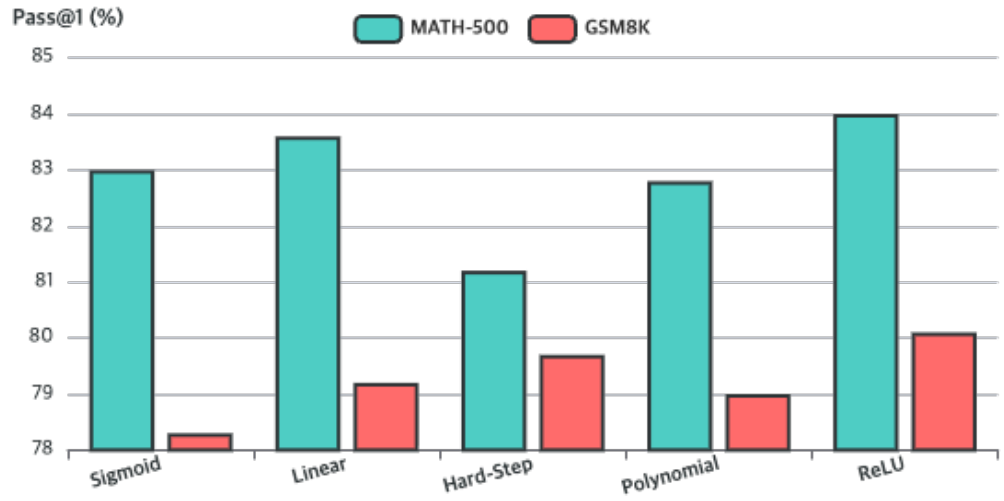
3. Hard-Step Gate
Abrupt binary transition

4. Polynomial-Fitted Gate
Custom polynomial curve

5. ReLU-Shaped Gate
Piecewise linear with threshold

All variants achieve effective performance

Gating Function Comparison (R1-1.5B)



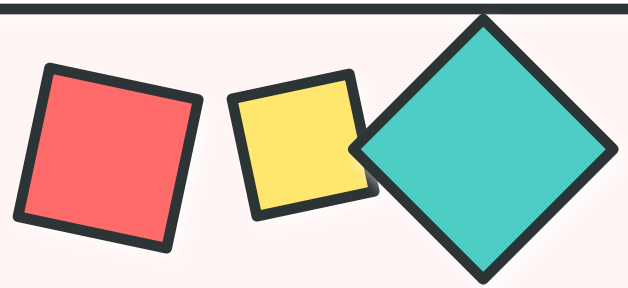
Key Finding

Results across multiple models, difficulty levels, and domains indicate that **all variants achieve effective reasoning performance** with only minor differences. This demonstrates that our approach is robust to the particular choice of gating function.

CHAPTER 11

Conclusion & Future Directions

Summary and research outlook



CONTRIBUTIONS

Key Contributions Summary

1 Confidence as Signal

Identified that **confidence can serve as a continuous and reliable signal** for characterizing both overthinking and underthinking in LRMs.

Key Insight

High variance → Overthinking
High confidence + Low variance → Underthinking

Impact

Enables **fine-grained behavioral control** through continuous monitoring

Empirical foundation

2 REBALANCE Framework

Proposed **REBALANCE** , a training-free framework that dynamically steers LRM reasoning trajectories via confidence-based hidden state modulation.

Core Components

- Explicit reasoning state modeling
- Confidence-based steering vector
- Dynamic control function

Key Properties

- **Training-free** deployment
- **Plug-and-play** integration
- No extra forward passes

Technical innovation

3 Empirical Validation

Demonstrated improvements in both **inference efficiency and accuracy** across diverse models and tasks.

Scale

- **4 models** (0.5B to 32B)
- **9 benchmarks** across domains
- Math, Science, Commonsense, Code

Results

- Up to **+10.0 points** accuracy gain
- Up to **-35.4%** token reduction
- Outperforms all baselines

Strong empirical evidence

IMPACT

Practical Impact & Deployment

Practical Benefits

Training-Free

Enables **immediate deployment** without expensive retraining or fine-tuning

Plug-and-Play

Easy integration with **existing LRMs** without architectural changes

Strong Generalization

Reduces need for **task-specific tuning** across diverse domains

Dual Improvement

Simultaneous **accuracy improvement** and **token reduction**

Deployment Scenarios

Resource-Constrained Environments

Addresses real deployment constraints by reducing computational costs while maintaining or improving accuracy

Production Systems

Can be seamlessly integrated into existing inference pipelines without disrupting current workflows

Multi-Domain Applications

Single configuration works across math, science, commonsense, and coding tasks

NPU Compatibility

Validated on **NPU devices** using openPangu-Embedded-7B-V1.1, demonstrating balanced thinking capabilities across hardware platforms

FUTURE WORK

Future Research Directions

Multi-Modal Reasoning

Apply REBALANCE to **multi-modal reasoning scenarios** involving vision, audio, and text

Other Paradigms

Extend to **other reasoning paradigms** beyond chain-of-thought (e.g., tree-of-thought)

Speculative Decoding

Integration with **speculative decoding** for further efficiency gains

Adaptive Windows

Exploration of **adaptive window strategies** that adjust based on task complexity

Other Architectures

Investigation of confidence signals in **other model architectures** beyond transformers

Theoretical Analysis

Deeper **theoretical understanding** of confidence-reasoning dynamics

> REBALANCE opens new avenues for efficient and balanced reasoning in Large Reasoning Models

ICLR 2026 REBALANCE Balanced Thinking